

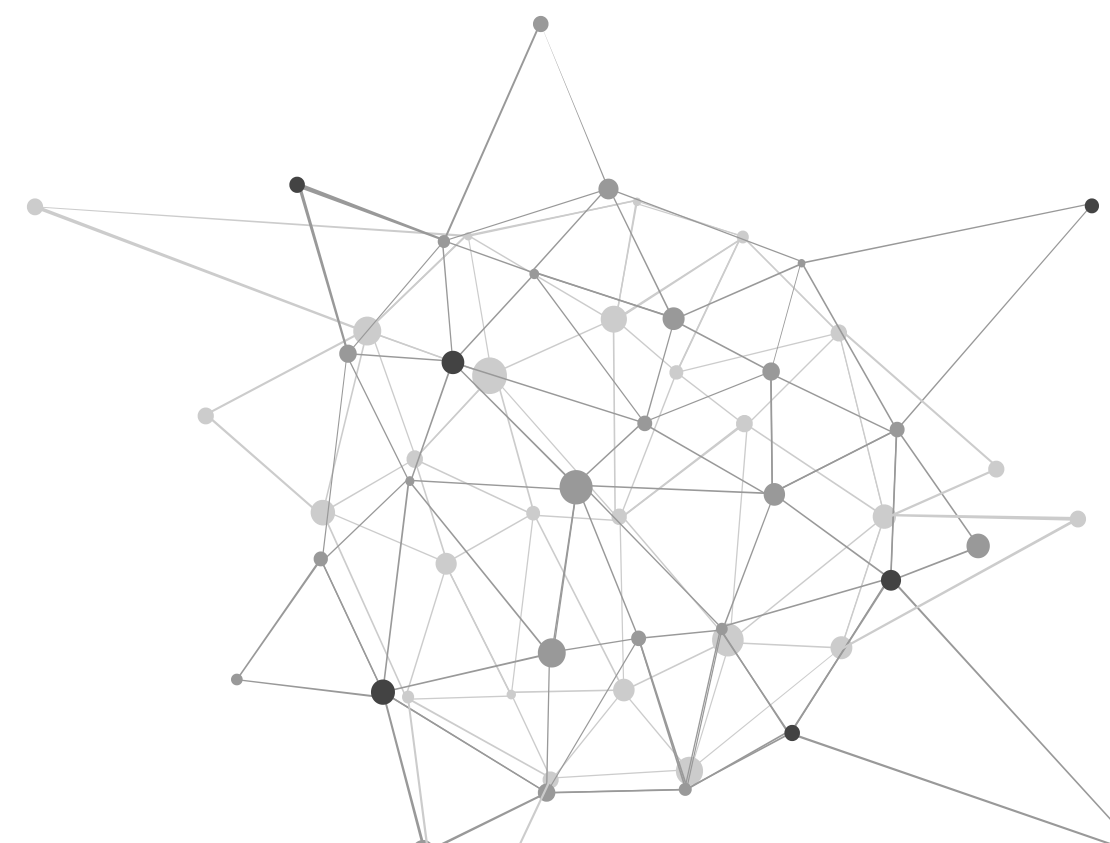


Detecting Intrusions, Revealing Anomalies

Machine learning and deep learning on CIC-IDS2017

Logistic Regression · LightGBM · Autoencoder

ESSA, DANA, RANA, RANEEM



Accuracy hides the attacks

Benign traffic dominates, so a model that flags nothing still looks good.

83.13%

accuracy of a model that labels
every flow as benign

0

of the 85,036 test attacks
detected by that same model

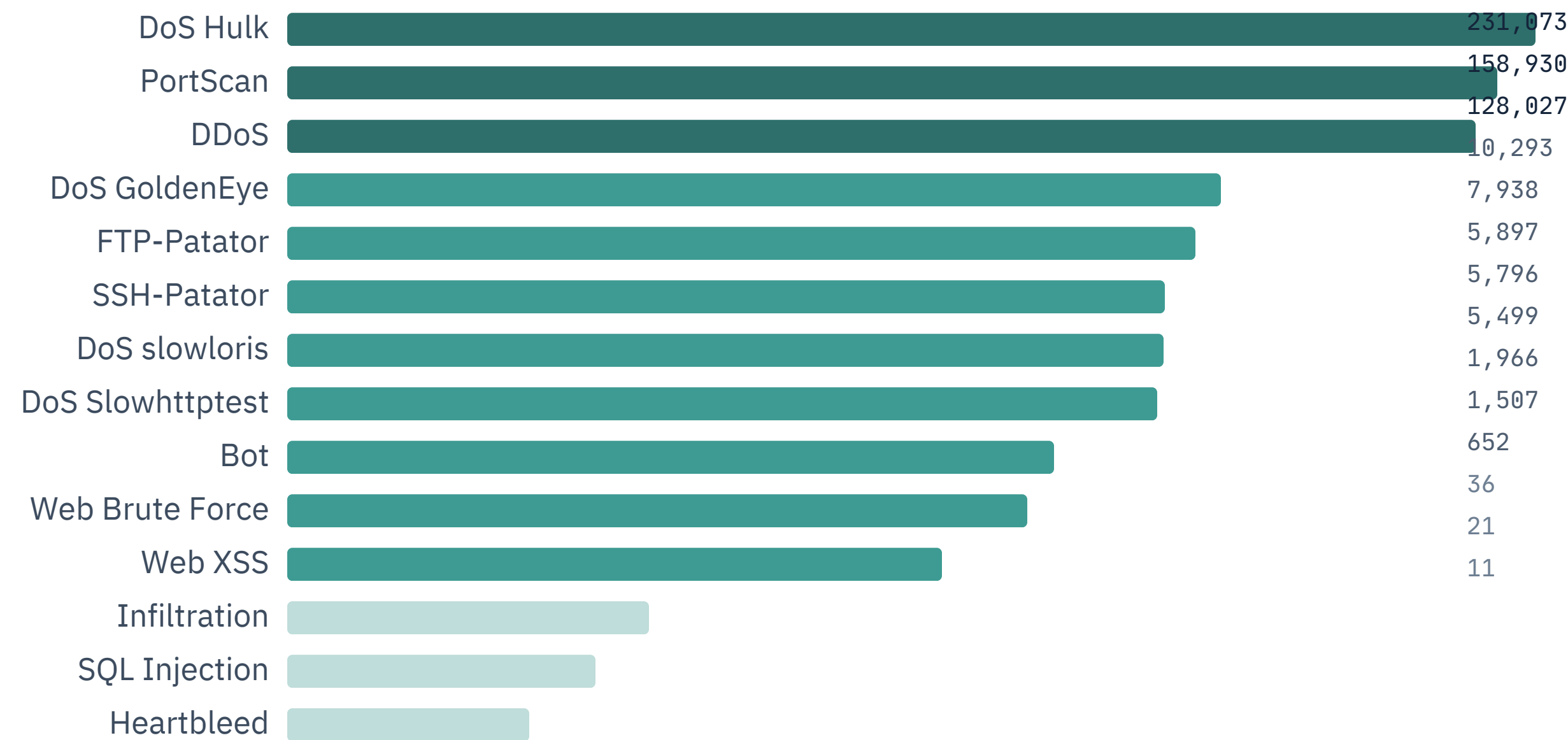
2.8M

flows in the dataset, 80% of
them perfectly normal

We therefore judge models by **AP, recall, missed attacks and false alarms** — never by accuracy.

CIC-IDS2017: fourteen attack families, very uneven

Eight capture files, 78 flow features, 2,830,743 flows. Bar length is on a logarithmic scale; counts are printed beside each bar.



Three families are 93% of all attack traffic. The grey three have 36, 21 and 11 flows — too few to support any per-family claim.

What the data taught us before modelling

Four findings from the EDA that change how the results must be read.

10.9%

of flows repeat an earlier feature vector; 698 patterns carry both labels

13

columns hold negative values; impossible ones set to missing, the -1 window sentinel kept

1 day

every attack family comes from a single capture file; Monday is benign only

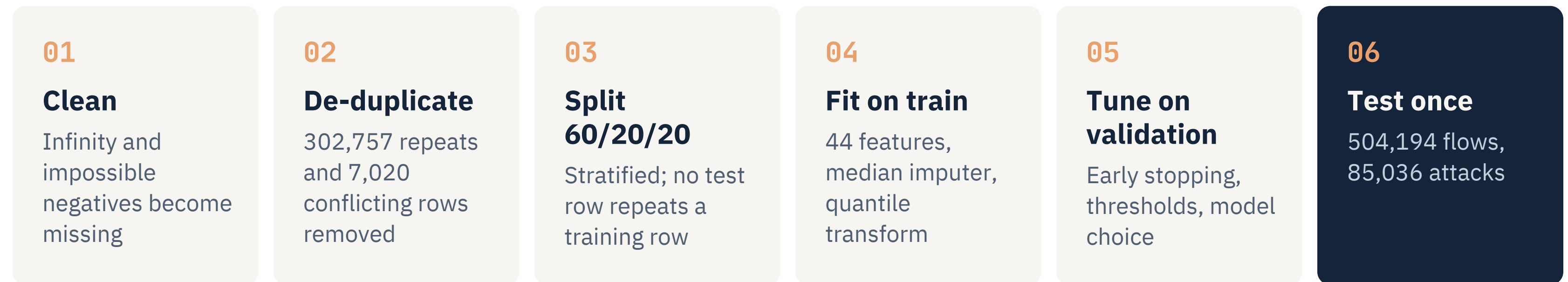
1 port

12 of 14 families target one port; Init_Win = -1 in 44% of benign, none of the attacks

The two dark cards are **shortcut risks**: a model can learn the capture setup instead of attack behaviour. We return to them twice.

A leak-free evaluation pipeline

Every decision is made on training or validation data. The test set is opened once.



An earlier version selected features on the full dataset and left duplicate flows spanning the split. This version fits every data-dependent step on training data only.

Three models, three ways of learning

From a linear baseline to an anomaly detector that never sees an attack.

LINEAR BASELINE

Logistic Regression

One straight boundary through the feature space. Class-weighted, quantile-scaled inputs. Tests how far the simple answer gets us.

GRADIENT-BOOSTED TREES

LightGBM

Hundreds of trees in sequence, each correcting the last. Early stopping on validation. Captures feature interactions a line cannot express.

ANOMALY DETECTION

Autoencoder

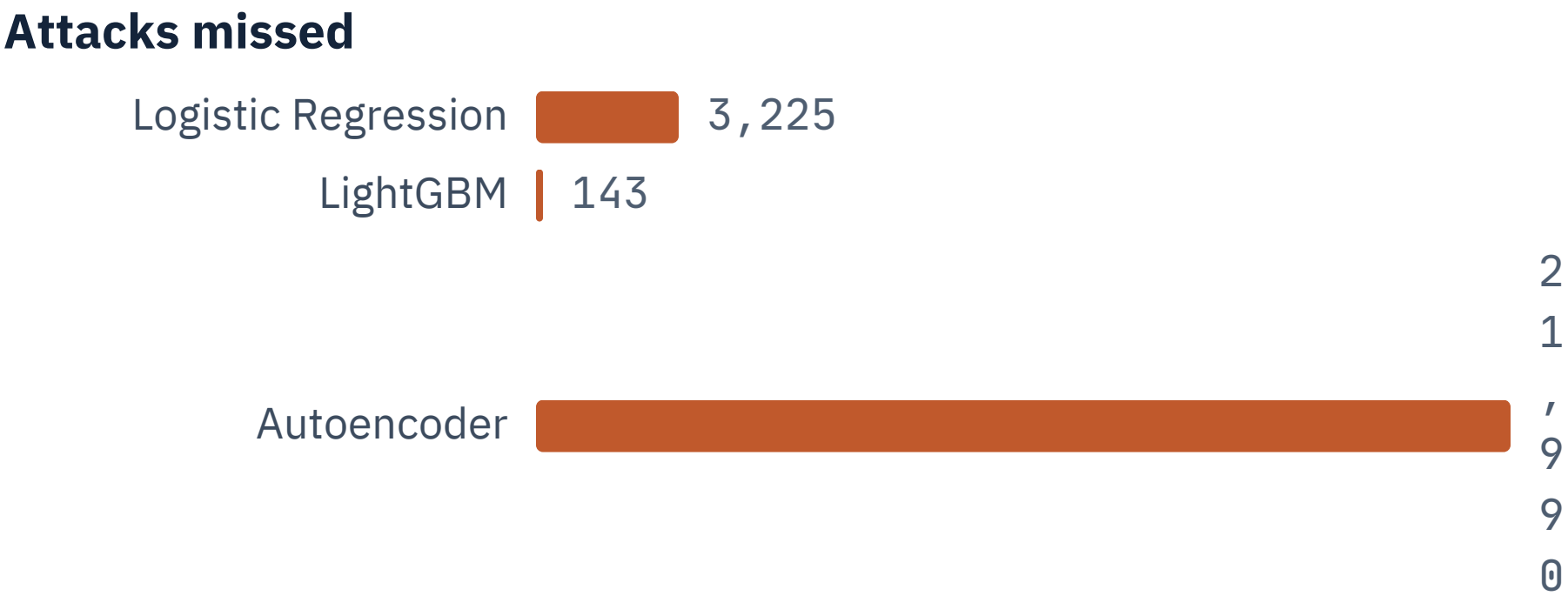
Trained on benign traffic only, 44-64-32-16-32-64-44. Never sees an attack. Reconstruction error is the anomaly score.

They were chosen to fail differently — which is what makes the comparison informative rather than a ranking.

LightGBM misses 143 of 85,036 attacks

Held-out test set: 504,194 flows, each model at its validation-tuned threshold.

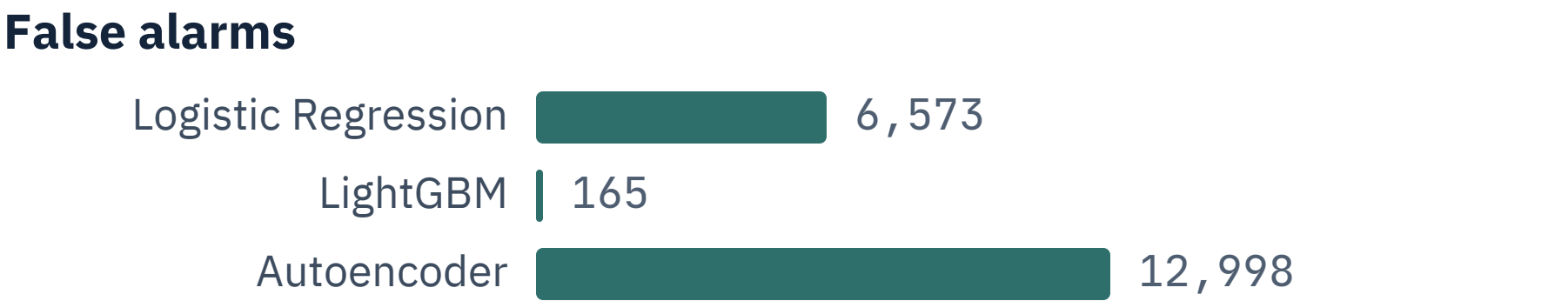
Attacks missed



Model	AP	Recall	FPR
Logistic Regression	0.9308	0.9621	1.57%
LightGBM	0.9999	0.9983	0.04%
Autoencoder	0.8511	0.7414	3.10%

LightGBM was selected on validation PR-AUC, before the test set was opened.

False alarms



The two weaker models fail in opposite directions: the baseline over-alerts, the Autoencoder under-detects.

Is 0.9999 too good to be true?

Four checks we ran before trusting the result.

0 rows

of the test set repeat a training row — asserted automatically after the split

0.000

train-versus-test PR-AUC gap for all three models: no memorisation

0.747

best ROC-AUC of any single feature alone: no feature leaks the label

Excluded

the label and capture-file columns are removed before any model sees the data

These rule out duplicate and single-feature leakage. They do **not** rule out capture shortcuts — which is why we ran the next experiment.

Bot traffic is LightGBM's blind spot

The aggregate recall of 0.9983 hides where the remaining errors sit.

72.7%

of Bot flows detected — the weakest per-family figure, from 399 test flows

109 of

143

about three quarters of everything the model misses is this one family

706

ports Bot traffic is spread across — the only family without a dominant port

No threshold recovers those flows. Bot is command-and-control traffic, built to look like routine browsing: short, periodic connections with no distinctive flow signature.

The models fail on different families

Test recall (%) for families with at least 100 test flows.

Attack family	Test flows	Log. Regression	LightGBM	Autoencoder
DoS Hulk	34,635	96.31	99.96	89.97
DDoS	25,632	99.84	99.97	78.86
PortScan	18,092	98.46	99.99	49.79
DoS GoldenEye	1,975	78.78	100.00	21.16
FTP-Patator	1,115	99.82	99.91	61.08
DoS slowloris	1,103	94.38	99.82	69.36
DoS Slowhttptest	990	89.80	100.00	42.22
SSH-Patator	635	13.70	99.69	57.01
Bot	399	0.00	72.68	3.26
Web Brute Force	297	79.46	98.99	0.00
Web XSS	149	84.56	100.00	0.00

Logistic Regression

0% on Bot, 13.7% on SSH-Patator — both encrypted or engineered to look ordinary

LightGBM

Strong everywhere except Bot

Autoencoder

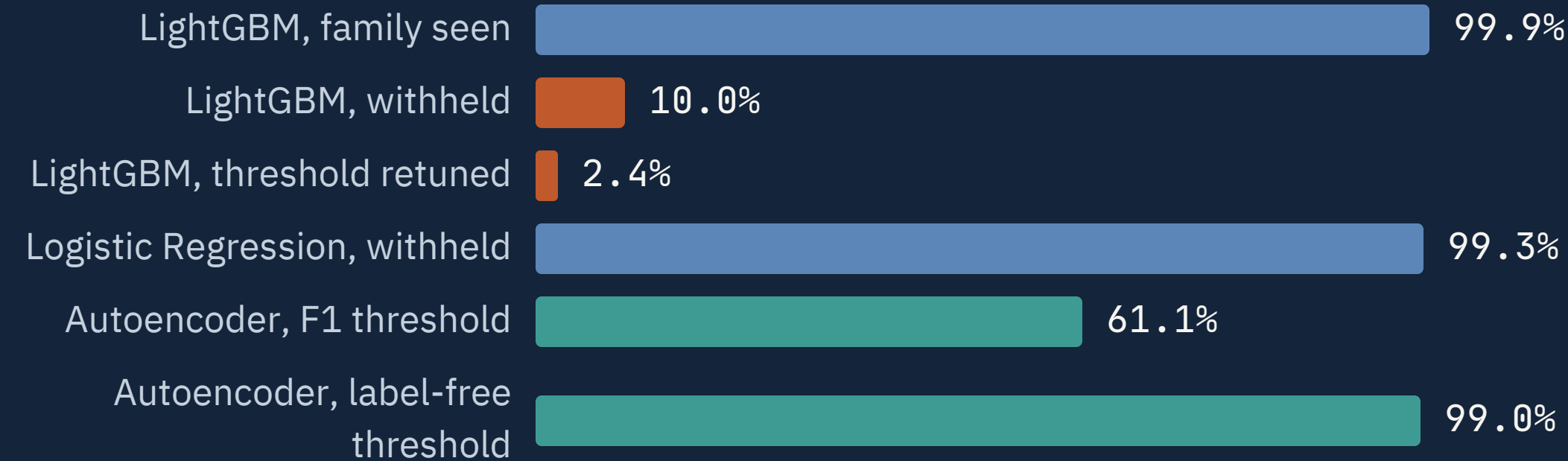
0% on web attacks, 21% on DoS GoldenEye — it detects statistically distinctive traffic, not attacks as such

Different blind spots are the reason we pair a supervised model with an anomaly detector.

Remove one attack family, and LightGBM collapses

FTP-Patator withheld from classifier training and from the validation data used to stop it.

Recall on FTP-Patator test flows



The label-free threshold

uses no attack labels at all, which is the only setting available for a genuinely new attack. It costs 21,226 false alarms, an FPR of 5.06%.

Logistic Regression barely moves

but raises roughly forty times LightGBM's false alarms, so part of its coverage is breadth, not recognition.

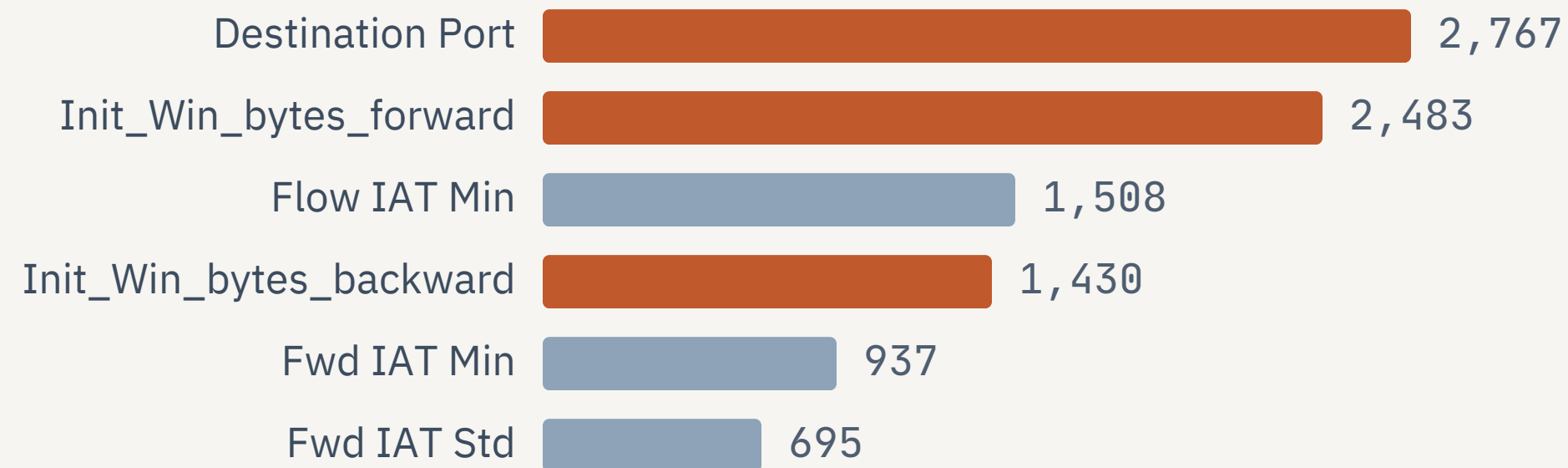
Caveat

preprocessing was fitted before the family was removed, so a fully unseen attack would likely drop further.

Why? The model leans on capture shortcuts

LightGBM's most-used features are exactly the ones the EDA flagged.

Times used to split a tree, top six features



Orange bars are the two capture-specific features.

Port stands in for attack type

FTP-Patator uses port 21 in 99.99% of its flows. A model that has never seen an attack on port 21 has no reason to flag it.

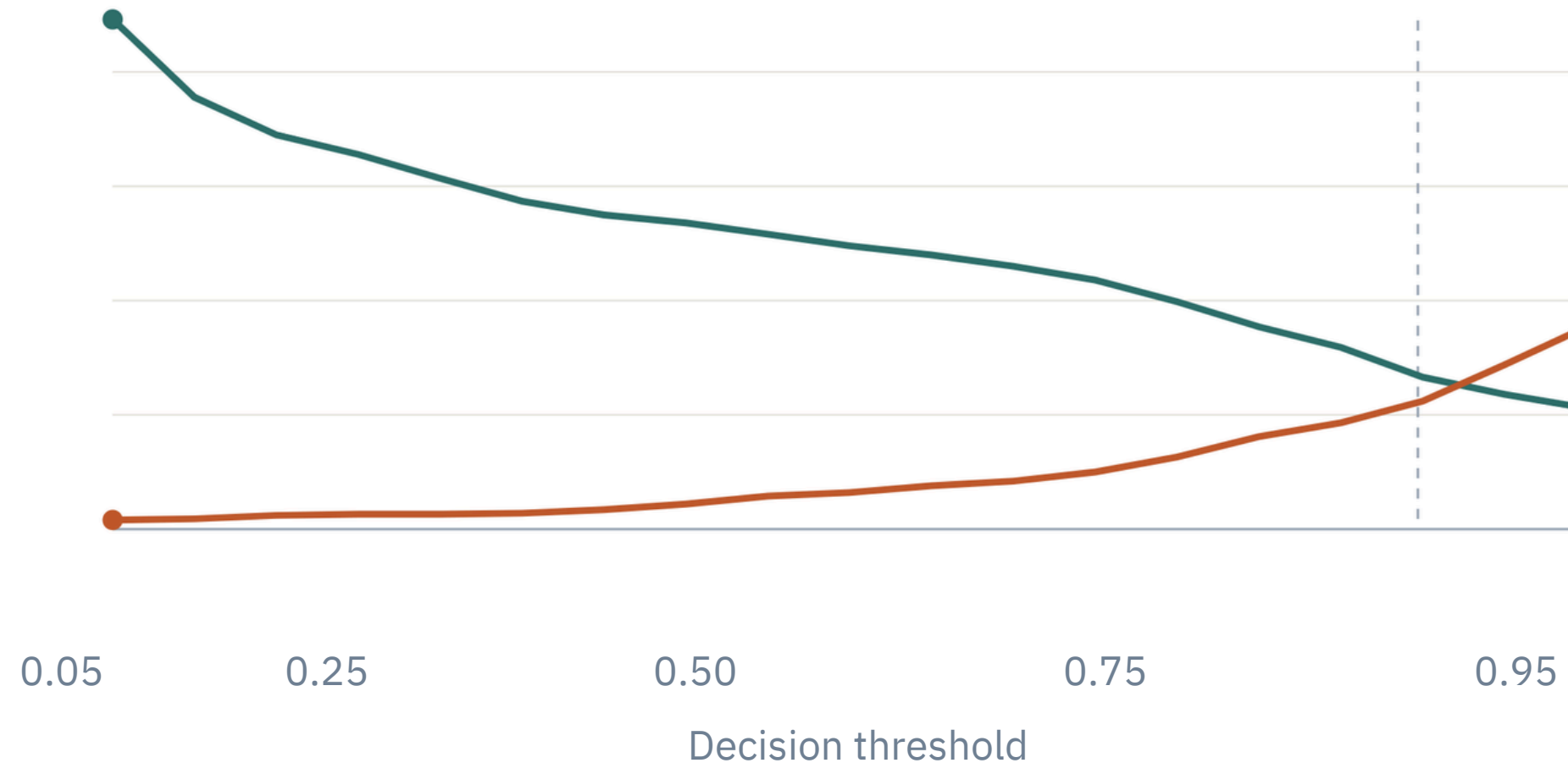
Init_Win = -1 stands in for benign

The value marks flows with no TCP window, mostly DNS. It separates protocol, not intent.

This is a hypothesis, not a proof: repeating the exclusion without these two features would test it directly.

Moving the threshold trades misses for alarms

LightGBM on the validation set, swept from 0.05 to 0.95 in nineteen steps.



False alarms

446 at the lowest threshold, 106 at the highest

Missed attacks

8 at the lowest threshold, 177 at the highest

F1 stays near 0.998

across the whole range, varying by 0.0013. F1 alone hides the choice that actually matters.

Two detectors, one policy decision

LightGBM as the primary filter, the Autoencoder as a safety net for unseen attacks.

Primary: LightGBM

Few false alarms on attack types it has seen. 165 alerts across 419,158 benign flows is reviewable by one analyst.

Safety net: Autoencoder

Flags traffic unlike anything benign, with no attack labels. It covers the failure mode the supervised model cannot.

Choosing the threshold is a security decision

Operating point	Threshold	Missed	False alarms
Balanced (F1)	0.847	143	165
Recall-oriented (F2)	0.329	22	341

The F2 point catches **121 more attacks for 176 more alerts** — about one and a half extra alerts per attack caught, still manageable for one analyst. We recommend it.

F1 treats a missed intrusion and a false alarm as equally costly. In security they are not, so the threshold is a policy choice, not a technical one.

Limitations and next steps

What these results can and cannot claim.

Limitations

- Random split within one 2017 capture: the same days and sessions appear in train and test
- Shortcut features inflate in-capture scores and may not transfer
- Family exclusion covers one family, and preprocessing had already seen it
- Autoencoder results vary by run and machine
- Heartbleed, SQL Injection and Infiltration have too few test flows for any claim

Future work

- Repeat family exclusion without the port and window features, and on more families
- A fully refitted pipeline for a true unseen-attack test
- Grouped or chronological evaluation, and a second dataset from another network
- Score-level fusion of LightGBM and the Autoencoder
- Sequence features for botnet command-and-control traffic

Every limitation on the left has a matching experiment on the right. None of them is a flaw in the implementation.

Reaching 99.9% on CIC-IDS2017 is easy. Knowing why, and when it fails, is the result.

143

attacks missed out of 85,036 — three quarters of them one family

99.9 to 10

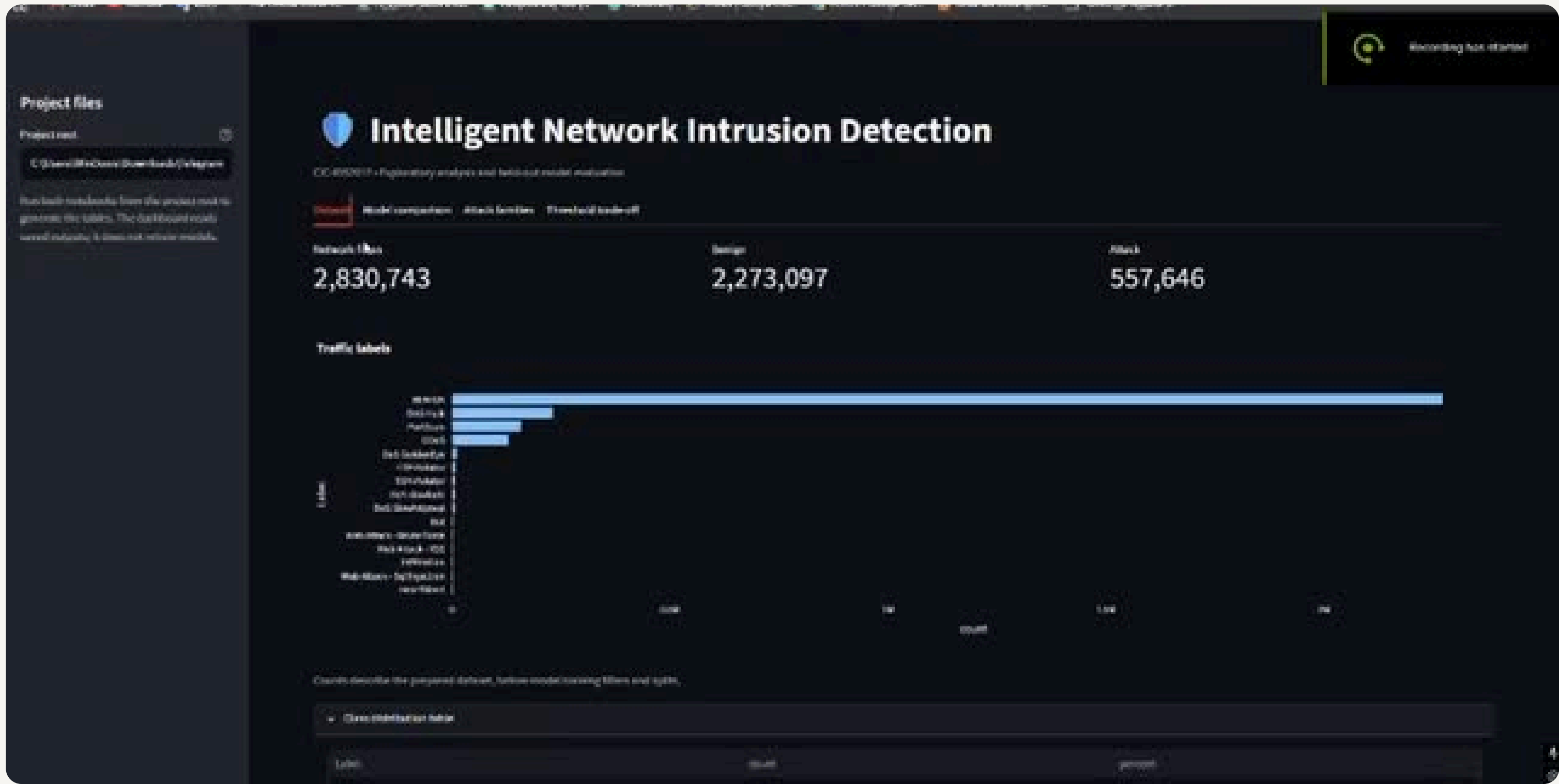
recall when one attack family is withheld from training

2 models

a supervised filter plus an anomaly safety net

An interactive view of the same results:

A Streamlit dashboard built on the saved notebook outputs: dataset composition across the fourteen attack families, the three models compared on detections, misses and false alarms, per-family recall, and a threshold slider that shows missed attacks trading against false alarms in real time.



Intelligent Network Intrusion Detection

Thank you
Questions?

APPENDIX A1

Model configuration

Logistic Regression

- class_weight = balanced
- solver = lbfgs, max_iter = 1000
- converged in 220 iterations
- input: imputed and quantile-transformed

LightGBM

- learning_rate 0.05, num_leaves 63
- min_child_samples 50
- subsample 0.8, colsample 0.8
- reg_alpha 0.1, reg_lambda 0.1
- scale_pos_weight 4.93
- early stopping: 320 of 1,000 trees

Autoencoder

- 44-64-32-16-32-64-44, ReLU, sigmoid output
- BatchNorm and dropout 0.1
- Adam (lr 0.001), MSE loss
- batch 512, up to 30 epochs
- early stopping, patience 5
- trained on 1,257,471 benign flows

Recall by family, all fourteen

Attack family	Test flows	Log. Regression	LightGBM	Autoencoder
DoS Hulk	34,635	96.31	99.96	89.97
DDoS	25,632	99.84	99.97	78.86
PortScan	18,092	98.46	99.99	49.79
DoS GoldenEye	1,975	78.78	100.00	21.16
FTP-Patator	1,115	99.82	99.91	61.08
DoS slowloris	1,103	94.38	99.82	69.36
DoS Slowhttptest	990	89.80	100.00	42.22
SSH-Patator	635	13.70	99.69	57.01
Bot	399	0.00	72.68	3.26
Web Brute Force	297	79.46	98.99	0.00
Web XSS	149	84.56	100.00	0.00
Infiltration *	7	0.00	57.14	85.71
Web SQL Injection *	6	16.67	83.33	0.00
Heartbleed *	1	0.00	100.00	100.00

* Fewer than ten test flows. A single flow moves the score by up to a hundred points, so these three are excluded from our detection claims.

Supporting experiments and data accounting

Autoencoder bottleneck width

Validation PR-AUC, single seed, exploratory

Bottleneck	Validation AP
8 units	0.9013
16 units (used)	0.8498
24 units	0.8657
32 units	0.8381

The 16-unit setting was fixed before this check was run. The ordering reversed between runs, so the differences are within run-to-run noise.

Data accounting

Step	Rows
Raw flows, 8 CSV files	2,830,743
Repeated copies removed	302,757
Label-conflicting rows removed	7,020
Distinct flow patterns	2,520,966
Train	1,512,579
Validation	504,193
Test	504,194

The attack rate falls from 19.70% to 16.87%, because duplicates are not spread evenly across classes.